

Cleware Web UI Anleitung

I. Gebrauchsanleitung

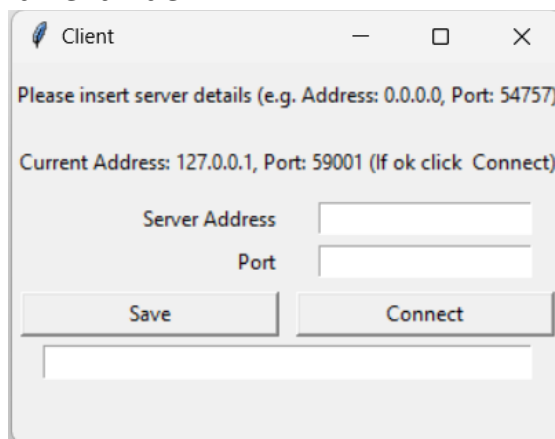
1- (Optional) Programordner öffnen und in 'ClewareUSB.ini' Host und Port anpassen soweit notwendig:

- Host = Adresse des Servers z.B.: 'XXXXXXX.ad005.onehc.net'
- Port = 59001
- Dll = USBaccessX64.dll

In der Datei ist die Adresse auf **localhost** (127.0.0.1) eingestellt und kann so gelassen werden.

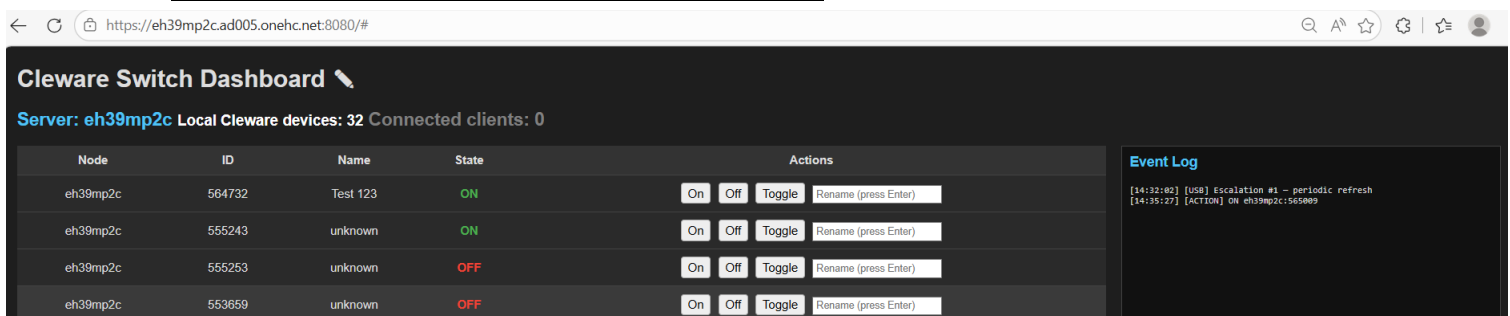
2- 'ClewareUSBServer.exe' auf PowerPC ausführen

3- 'ClewareUSBClient.exe' auf Clients mit USB Switches starten. Serverinformationen eingeben, speichern und verbinden.



Client UI

4- Im Browser öffnen (Server): <https://<Serveradresse>:8080> -->z.B.:
<https://EH39M31C.ad005.onehc.net:8080>



Weboberfläche

5- Beschreibung der Web UI

The screenshot shows the 'Cleware Switch Dashboard' interface. At the top, there's a header with a pencil icon for editing the page title. Below the header, a status bar displays 'Server: eh39mp2c', 'Local Cleware devices: 32', and 'Connected clients: 0'. The main content area features a table of devices and an 'Actions' panel on the right.

Node	ID	Name	State
eh39mp2c	564732	Test 123	ON
eh39mp2c	555243	unknown	ON
eh39mp2c	555253	unknown	OFF
eh39mp2c	553659	unknown	OFF

The 'Actions' panel on the right contains four rows of controls, each with 'On', 'Off', 'Toggle', and 'Rename (press Enter)' buttons.

Seitentitel ändern

Serverinformationen sowie Anzahl der angeschlossenen Geräte und Clients

Node stellt dar auf welchen PC ein Gerät angeschlossen ist

IDs, Namen und Zustände der einzelnen Geräte

Aktionen:
On: Gerät anschalten
Off: Gerät abschalten
Toggle: Gerätzustand ändern
Rename: Geräte name anpassen

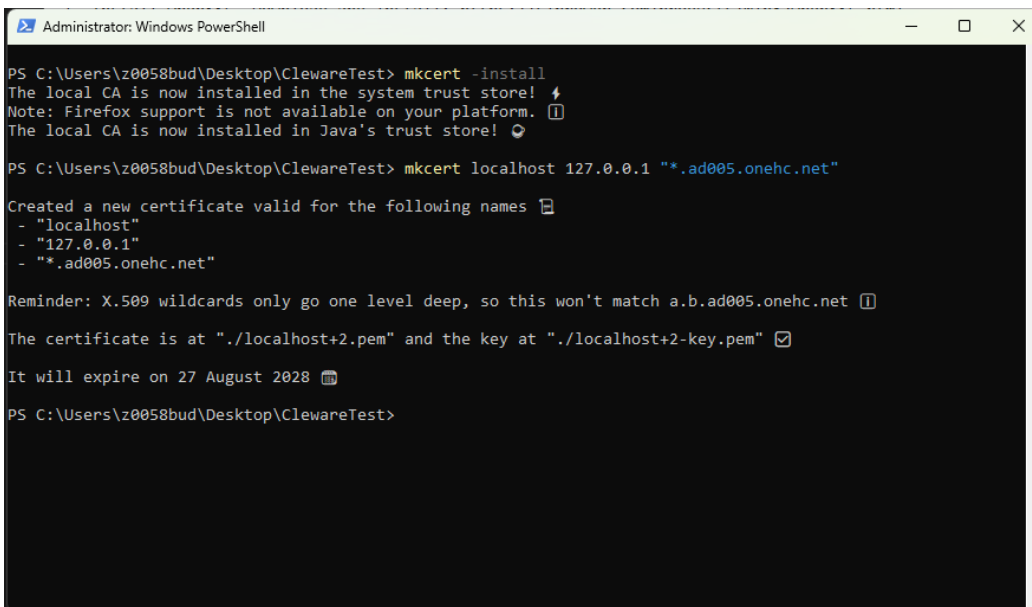
Event Log

```
[14:32:02] [USB] Escalation #1 - periodic refresh  
[14:35:27] [ACTION] ON eh39mp2c:565809
```

Event Log
Listet alle Ereignisse und Aktionen auf, die auf dem Server oder auf verbundene Clients stattfinden

6- Wichtig! HTTPS/SSL Zertifikat generieren (falls abgelaufen):

- 1) Powershell als Admin ausführen
- 2) Mkcert installieren:
 - Eingeben und ausführen: <https://community.chocolatey.org/install.ps1>
- 3) Programverzeichnis öffnen (cd C:\...)
- 4) mkcert -install
- 5) mkcert localhost 127.0.0.1 "*0ad005.onehc.net"
- 6) Generierte '.pem' und '.key' Dateien gleichermaßen wie die alten umbenennen, in den Programordner einfügen bzw. Die alten ersetzen.



```
Administrator: Windows PowerShell

PS C:\Users\z0058bud\Desktop\ClewareTest> mkcert -install
The local CA is now installed in the system trust store! ⚡
Note: Firefox support is not available on your platform. ⓘ
The local CA is now installed in Java's trust store! ☺

PS C:\Users\z0058bud\Desktop\ClewareTest> mkcert localhost 127.0.0.1 "*.ad005.onehc.net"

Created a new certificate valid for the following names ⓘ
- "localhost"
- "127.0.0.1"
- "*.ad005.onehc.net"

Reminder: X.509 wildcards only go one level deep, so this won't match a.b.ad005.onehc.net ⓘ

The certificate is at "./localhost+2.pem" and the key at "./localhost+2-key.pem" ⓘ

It will expire on 27 August 2028 ⓘ

PS C:\Users\z0058bud\Desktop\ClewareTest>
```

7- Hinweise

Es können laut Cleware bis zu 32 Geräte gleichzeitig angeschlossen werden. Das ist gemäß die durchgeführten Tests tatsächlich möglich, hängt aber stark vom System ab. Z.B. ein Workstation eignet sich hierfür besser als ein ThinClient. Außerdem damit die Stabilität im Dauerbetrieb gegeben ist, müssen die USB-Geräte einzeln mit Strom versorgt werden, z.B. durch USB-Hub mit eigenem Stromkabel.

Bei erster Ausführung des Programs auf einem PC ist es empfohlen eine **Inbound Regel** im Firewall anzulegen und der verwendete Port zulassen, damit eine Verbindung außerhalb des Arbeitsplatzes möglich ist: Firewall öffnen -> Netzwerkschutz -> Erweiterte Einstellungen -> Inbound Rules.

Das Program als Startup Program festlegen, damit es nach Startup automatisch ausgeführt wird.

II. Technische Beschreibung: Code

Die Anwendung wurde mit Python programmiert und der Code für das Program besteht aus 3 Python Dateien.

- **ClewareUSBServer.py** für das Serverprogram
- **ClewareUSBClient.py** für das Clientprogram
- **ClewareUSBLib.py** für die Schnittstelle durch die USBaccessX64.dll-Datei. Diese Datei wird im Serverprogram und im Clientprogram importiert und ist daher sehr wichtig.

1. ClewareUSBServer.py: Serverprogram

Das Serverprogram beinhaltet alle Komponenten für die Steuerung der Power Switches sowie alle Komponente der Web UI.

Für die Anwendung werden folgende Bibliotheken benötigt:

```
import json
import socket
import ssl
import threading
import time
import os
import re
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from queue import Queue
from typing import Dict, Tuple, List
from datetime import datetime
from urllib.parse import parse_qs, urlparse
from ClewareUSBLib import *
```

Konfigurationsparameter und globale Parameter werden am Anfang definiert. Z.B. Anzahl der connected Clients, Serbvername, Device counter, usw...

Event Log wird mit der Methode `def log_event()` implementiert.

```
# ===== LOGGING =====
def log_event(msg: str):
    entry = f"[{datetime.now().strftime('%H:%M:%S')}] {msg}"
    print(entry)
    with EVENT_LOG_LOCK:
        EVENT_LOG.append(entry)
        if len(EVENT_LOG) > MAX_EVENTS:
            EVENT_LOG[:] = EVENT_LOG[-MAX_EVENTS:]
```

Der **USB_Worker** nimmt die Anweisungen des Nutzers („On“, „Off“, „Toggle“,...) entgegen und führt die aus. Für mehr Stabilität wird der USB_Worker in einem separaten Thread ausgeführt.

```

# ===== USB WORKER =====
class USBCommand:
    def __init__(self, cmd, devID=None, extra=None):
        self.cmd = cmd
        self.devID = devID
        self.extra = extra
        self.result = None
        self.event = threading.Event()

def usb_worker():
    global USB_IN_RECOVERY

    while True:
        job = USB_QUEUE.get()

        with USB_EXEC_LOCK:
            try:
                if job.cmd == "list":
                    job.result = cwUSB_list_Devices()

                elif job.cmd == "state":
                    job.result = str(cwUSB_get_StateFromNum(job.devID))

                elif job.cmd == "set":
                    cwUSB_set_StateToNum(job.devID, job.extra)
                    job.result = "OK"

                elif job.cmd == "rename":
                    cwUSB_set_NametoNum(job.devID, job.extra)
                    job.result = "OK"

                elif job.cmd == "usb_type":
                    job.result = cwUSB_get_USBType(job.devID)

                elif job.cmd == "watchdog":
                    if not USB_IN_RECOVERY:
                        cwUSB_CalmWatchdog(job.devID, 1, 0)

            except Exception as e:
                USB_IN_RECOVERY = False
                job.result = f"ERROR:{e}"

        job.event.set()

def usb_execute(cmd, devID=None, extra=None):
    if USB_QUEUE.qsize() > MAX_USB_QUEUE:
        return "BUSY"

    job = USBCommand(cmd, devID, extra)

    USB_QUEUE.put(job)

    if cmd in ("recover", "reset"):
        if USB_IN_RECOVERY:
            return None

    job.event.wait(timeout=10)
    return job.result

```

Die Methoden **usb_escalating_recover()** und **watchdog_loop()** stellen Sicher, dass die Geräte bzw. Das Program stabil laufen.

Der Abschnitt **TCP (send_msg(), recv_msg(), rpc_call())** steuert die Kommunikation und Datenaustausch zwischen Server und Clients.

```

# ===== TCP =====
def send_msg(sock, msg):
    sock.sendall((msg + "\n").encode())

def recv_msg(sock):
    buffer = ""
    try:
        while True:
            data = sock.recv(1024).decode()
            if not data:
                return None
            buffer += data
            if "\n" in buffer:
                msg, _ = buffer.split("\n", 1)
                return msg.strip()
    except Exception as e:
        log_event(f"[RPC] recv_msg failed: {e}")
        return None

def rpc_call(sock, msg):
    try:
        send_msg(sock, msg)
        return recv_msg(sock)
    except Exception as e:
        log_event(f"[RPC] {msg!r} failed: {e}")
        return None

```

In dem Abschnitt **STATE LOOP** werden die Zustände der Geräte kontinuierlich gelesen und aktualisiert. **Extract_dev(line)** liest aus dem Regsiter die Geräte IDs. **Extract_name(line)** liest aus dem Register die Gerätenamen. State_loop() aktualisiert die Gerätezustände und erfasst eventuelle Fehler. Für mehr Stabilität werden die

Geräte Namen, IDs und Zustände in einem Cache gespeichert. Der State_Loop wird in einem separaten Thread ausgeführt.

```
# ===== STATE LOOP =====
def extract_dev(line):
    m = re.search(r"serial number=\s*(\d+)", line)
    return int(m.group(1)) if m else None

def extract_name(line):
    m = re.search(r"Name=(.*)$", line)
    return m.group(1).strip() if m else ""

def state_loop():
    global USB_HEALTH_ERRORS
    global LAST_USB_RECOVERY
    global device_counter
    global conn_clients

    while True:
        cycle_ok = True
        last_bad_dev = None
        device_counter = 0
        conn_clients = 0

        txt = usb_execute("list")

        if USB_IN_RECOVERY:
            time.sleep(1)
            continue

        if not txt:
            cycle_ok = False
        else:
            lines = [l.strip() for l in txt.splitlines() if l.strip()]
            if not lines:
                cycle_ok = False
            else:
                for line in lines:
                    devID = extract_dev(line)
                    if devID is not None:
                        if devID is not None:
                            device_counter += 1

                        if not txt or not lines:
                            usb_escalating_recover(reason="device list empty")

                        if devID is None:
                            continue

                        name = extract_name(line)
                        raw_state = usb_execute("state", devID)

                        if raw_state not in ("0", "1"):
                            cycle_ok = False
                            last_bad_dev = devID
                            break # stop scanning - failure detected

                        with STATE_CACHE_LOCK:
                            STATE_CACHE[(server_name, devID)] = raw_state
                            key = (server_name, devID)
                            if name != "unknown":
                                DEVICE_NAME_CACHE[key] = name
                            elif key not in DEVICE_NAME_CACHE:
                                DEVICE_NAME_CACHE[key] = "unknown"

        # ---- health evaluation (ONCE per cycle) ----
        if cycle_ok:
            USB_HEALTH_ERRORS = 0
        else:
            USB_HEALTH_ERRORS += 1

        # ---- escalation based on errors ----
        if USB_HEALTH_ERRORS >= USB_HEALTH_THRESHOLD:
            log_event("USB health degraded - escalating recovery")
            usb_escalating_recover(last_bad_dev)
            USB_HEALTH_ERRORS = 0
            LAST_USB_RECOVERY = time.time()
```

execute_cmd(node, dev, action, extra=None) nimmt die Commands von den verschiedenen nodes(Server oder Clients) und führt die mit der **usb_execute()** Methode aus.

Der Abschnitt **Save page title** ist für die Speicherung des Seitentitels zuständig, wenn der Name geändert wird. Seitentitel wird in ClewareUSB.ini gespeichert.

Der Abschnitt **WEB** stellt die Implementierung Web UI dar. Innerhalb der Klasse **Handler(BaseHTTPRequestHandler)** wird die gesamte Web Oberfläche implementiert. Informationen wie Geräte IDs, Namen und Zustände werden in Echtzeit angezeigt. Dafür wurde zusätzlich zu HTML SSE durch **sse_broadcaster()** implementiert.

Der Abschnitt **SERVER (accept_loop(sock))** steuert die Verbindung zwischen Server und Clients.

```
# ===== SERVER =====
def accept_loop(sock):
    global conn_clients

    while True:
        #conn_clients = 0
        try:
            conn, addr = sock.accept()
            name = recv_msg(conn)

            if not name or not name.startswith("HELLO "):
                conn.close()
                continue

            cname = name.split()[1]

            with connected_lock:
                connected_clients[cname] = (conn, addr)

            log_event(f"Client connected: {cname}")
            #conn_clients += 1

        except:
            time.sleep(0.1)
```

In Main werden die Geräte gelesen, die Threads ausgeführt, Serverinformationen gelesen und der Webserver gestartet. Die SSL bzw. HTTPS Funktionalität für eine sichere Navigation ist hier auch durch das Einlesen der Zertifikatsdateien (**localhost+1.pem**, **localhost+1-key.pem**) gegeben. Um die Zertifikate zu erneuern, siehe oben.

```
# ===== MAIN =====
def main():
    global server_name

    server_name = socket.gethostname().lower()

    cwUSB_setup()
    #print("Cleware devices found:", cwNoOfDevices)

    #start threads AFTER setup
    threading.Thread(target=usb_worker, daemon=True).start()
    threading.Thread(target=state_loop, daemon=True).start()
    threading.Thread(target=watchdog_loop, daemon=True).start()

    host, port, _ = cwUSB_getConfig()

    s = socket.socket()
    s.bind(("", port))
    s.listen()

    threading.Thread(target=accept_loop, args=(s,), daemon=True).start()

    web = ThreadingHTTPServer(("0.0.0.0", WEB_PORT), Handler)

    # Enable SSL
    ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    ctx.load_cert_chain("localhost+1.pem", "localhost+1-key.pem")
    web.socket = ctx.wrap_socket(web.socket, server_side=True)

    web.serve_forever()

if __name__ == "__main__":
    main()
```

2- ClewareUSBClient.py: Client Program

Das Client Program besteht aus einer GUI und eine Konfigurationsdatei: **ClewareClientConfig.ini**, in der die Serverinformationen gespeichert sind(host, port, dll).

Wie der Server, das Client Program beinhaltet ein **USB WORKER**.

Server Infomationen werden durch die Methode **get_config()** von der Konfigurationsdatei gelesen.

Save_config() wird verwendet, um die Serverinformationen in die .ini-Datei zu speichern für zukunfftige Verbindungen.

```
def get_config():
    config = configparser.ConfigParser(allow_unnamed_section=True)
    config.read('ClewareClientConfig.ini')
    host = '0.0.0.0' # Default host
    port = 0        # Default port
    dll = r"USBAccessX64.dll"
    try:
        NetConfig = config[configparser.UNNAMED_SECTION]
        host = NetConfig.get('host', host)
        port = NetConfig.getint('port', port)
        dll = NetConfig.get('dll', dll)
    except Exception:
        pass
    return [host, port, dll]

def save_config():
    config = configparser.ConfigParser(allow_unnamed_section=True)
    config.read('ClewareClientConfig.ini')

    host = ServerAddress.get().strip()
    port = Port.get().strip()
    if not host or not port:
        set_status("Please provide host and port", "lightcoral")
        return
    try:
        int(port)
    except ValueError:
        set_status("Port must be an integer", "lightcoral")
        return
    with open("ClewareClientConfig.ini", "w") as f:
        f.write(f"host = {host}\nport = {port}\ndll = USBAccessX64.dll")
    set_status("Connection info saved", "lightgreen")
```

Der Client wird durch die Methode **run_agent()** gestartet und USB Geräte werden mit **cwUSB_steup()** einmal initialisiert.

```
def run_agent():
    global isRunning

    host, port, _ = get_config()
    node = socket.gethostname().lower()

    print(f"[CLIENT] Starting Cleware agent as {node}")

    # Initialize USB once
    cwUSB_setup()

    delay = RECONNECT_BASE_DELAY

    while True:
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
                s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
                s.connect((host, port))
                s.sendall(f"HELLO {node}\n".encode())

                isRunning = True
                print("[CLIENT] Connected to server")

                delay = RECONNECT_BASE_DELAY

                while True:
                    data = s.recv(4096)
                    if not data:
                        raise ConnectionError("Server disconnected")

                    line = data.decode(errors="ignore").strip()
                    if not line:
                        continue

                    parts = line.split()
                    cmd, args = parts[0], parts[1:]

                    resp = usb_execute(cmd, args)
                    # debug: log what client will send back
                    #print(f"[CLIENT] RESP {cmd!r} -> {resp!r}")
                    s.sendall((resp + "\n").encode("utf-8"))

        except Exception as e:
            print(f"[CLIENT] Disconnected: {e}")
            isRunning = False
            time.sleep(delay)
            delay = min(delay * 2, RECONNECT_MAX_DELAY)
            break # Exit after one failed attempt for better UX in this demo
```


Connect() stellt die Verbindung mit dem Server her.

```
def connect():

    global isRunning
    isRunning = False # Reset before trying to connect

    def agent_thread():
        run_agent()

    set_status("Connecting...", "khaki")
    threading.Thread(target=agent_thread, daemon=True).start()

    # Wait for connection attempt to finish (success or failure)
    for _ in range(30): # Wait up to ~3 seconds (30 x 0.1s)
        if isRunning:
            set_status("Connected", "lightgreen")
            print("Agent is running")
            break
        time.sleep(0.1)
    else:
        set_status("Connection failed", "lightcoral")
```

Der Abschnitt **GUI** enthält die Oberfläche des Cleints. Die GUI wurde mit der **tkinter** Bibliothek implementiert.

```
#####GUI#####
root = tk.Tk()
root.title("Client")

tk.Label(root, text=f"Please insert server details (e.g. Address: 0.0.0.0, Port: 54757)").grid(row=0, column=0, columnspan=2, pady=(8,10))
#tk.Label(root, text="").grid(row=1, column=0) # Spacer
tk.Label(root, text=f"Current Address: {get_config()[0]}, Port: {get_config()[1]} (If ok click Connect)").grid(row=2, column=0, columnspan=2, pady=(8,10))

tk.Label(root, text="Server Address").grid(row=3, column=0, sticky="e", padx=4, pady=2)
tk.Label(root, text="Port").grid(row=4, column=0, sticky="e", padx=4, pady=2)

ServerAddress = tk.Entry(root)
Port = tk.Entry(root)
ServerAddress.grid(row=3, column=1, padx=4, pady=2)
Port.grid(row=4, column=1, padx=4, pady=2)

button = tk.Button(root, text="Save", width=20, command=save_config)
button.grid(row=5, column=0, columnspan=1, pady=5)

button2 = tk.Button(root, text="Connect", width=20, command=connect)
button2.grid(row=5, column=1, columnspan=1, pady=5)

text_widget = tk.Text(root, height=1, width=35, state="disabled")
text_widget.grid(row=6, column=0, columnspan=2, pady=(0,8))

messageVar = tk.Label(root, text="", width=40)
messageVar.grid(row=7, column=0, columnspan=2, pady=(0,8))
```

3- ClewareUSBLib.py: Schnittstelle zu Cleware Geräte

In der **ClewareUSBLib.py** Datei werden die Cleware dll und der Registerpfad der Geräte eingelesen. Die DLL wird hier „übersetzt“, damit die Programme mit den Switches kommunizieren können. Die enthaltenen Funktionen werden in **ClewareUSBServer.py** und **ClewareUSBClient.py** importiert.

```
from ctypes import *
import time
import winreg
import sys
import configparser
import os

REG_PATH = r"SOFTWARE\Cleware GmbH\USB"

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DLL_PATH = os.path.join(BASE_DIR, "USBAccessX64.dll")

cwbInitialed = False

# load config from ini-file or provide default
def cwUSB_getConfig():
    config = configparser.ConfigParser(allow_unnamed_section=True)
    config.read('ClewareUSB.ini')
    # tHost = 'IMW1027359C.ad005.onehc.net'
    tHost = 'IMW1027359C.ad005.onehc.net'
    iPort = 59001
    tDll = r"USBAccessX64.dll"
    try:
        NetConfig = config[configparser.UNNAMED_SECTION]
        tHost = NetConfig.get('host', tHost)
        iPort = NetConfig.getint('port', iPort)
        tDll = NetConfig.get('dll', tDll)
    except Exception:
        pass
    return [tHost, iPort, tDll]
```